

Módulo 11 – Capítulo 04 – Sección 01

Modelos de tenancy: Silo, Pool y Bridge — trade-offs de aislamiento y costo

Cuando una plataforma de IA enterprise sirve a múltiples clientes, unidades de negocio, o departamentos, la primera decisión de arquitectura que determina todo lo demás es el modelo de tenancy: qué comparten los tenants y qué tienen separado. Esta decisión no es solo técnica — afecta directamente el modelo de negocio del producto (qué garantías de privacidad puede ofrecer), los acuerdos de cumplimiento con los clientes (los DPAs exigen niveles específicos de aislamiento de datos), y la complejidad operacional del equipo de plataforma (cuántas instancias de infraestructura deben mantenerse y actualizarse). Tomar esta decisión sin un framework explícito produce sistemas donde el aislamiento se asumió pero nunca se verificó, y donde los incidentes de cross-tenant data access se descubren en el peor momento posible.

El **modelo Silo** ofrece el máximo aislamiento técnico: cada tenant opera con infraestructura completamente dedicada. En el contexto de una plataforma de IA, esto significa un cluster de Kubernetes propio (o al menos un namespace completamente aislado con Network Policies), una base de datos vectorial propia o una instancia separada, y potencialmente una instancia propia de LLM si el tenant usa modelos self-hosted. Las ventajas son evidentes: el aislamiento es total, los SLOs pueden configurarse independientemente por tenant, y el cumplimiento de normas como HIPAA o requisitos de residencia de datos es más simple de demostrar. El costo también es evidente: la infraestructura se multiplica por el número de tenants, y las operaciones de mantenimiento (actualizaciones de seguridad, upgrades de versiones, rotación de claves) deben ejecutarse N veces en lugar de una.

El **modelo Pool** maximiza la eficiencia de recursos: todos los tenants comparten la misma infraestructura con aislamiento lógico implementado a nivel de aplicación. El mismo cluster de Kubernetes, la misma base de datos vectorial con namespaces separados, y el mismo endpoint de LLM sirven a todos los tenants. La reducción de costos de infraestructura respecto al modelo Silo es del 70-80%, y la complejidad operacional es significativamente menor. El riesgo central de este modelo es el aislamiento lógico implementado incorrectamente: si la lógica de tenant_id en la aplicación tiene un bug — por ejemplo, una

query vectorial que no incluye el filtro de namespace — puede exponer datos de un tenant a otro. El **noisy neighbor problem** es la segunda vulnerabilidad del modelo Pool: un tenant que ejecuta un job de indexación masiva puede saturar los recursos compartidos y degradar la latencia de todos los demás tenants activos en ese momento.

El **modelo Bridge** (también llamado tiered tenancy) combina los dos modelos de manera pragmática: infraestructura compartida (pool) para los tenants estándar que no tienen requisitos de aislamiento estrictos, y silo dedicado (opt-in o mandatorio) para tenants Enterprise o Premium que requieren garantías más fuertes por contrato, por sector regulado, o por volumen que hace el silo individualmente rentable. Este modelo es el que más directamente se alinea con la realidad comercial de las plataformas SaaS enterprise: el 80% de los clientes son tenants estándar que se sirven eficientemente desde el pool, y el 20% de los clientes (que generan el 60-70% de los ingresos) son clientes Enterprise que requieren y pagan por el aislamiento del silo.

Nota del Arquitecto: La decisión de modelo de tenancy debe tomarse antes de escribir la primera línea de código del sistema multi-tenant, porque cambiarla retroactivamente es una reescritura parcial. En particular, migrar de pool a silo para un tenant específico en respuesta a un requisito contractual puede ser una operación de semanas si el sistema no fue diseñado con esa capacidad desde el inicio. El modelo Bridge, diseñado desde el principio con la posibilidad de silo por tenant, evita este problema.

Trade-offs de cada modelo de tenancy

- **Modelo Silo:** aislamiento total (red, datos, cómputo), SLOs independientes por tenant, y cumplimiento simplificado; costo operacional: N veces el costo de un tenant único, con complejidad de mantenimiento proporcional al número de tenants.
- **Modelo Pool:** reducción de costos del 70-80% respecto al silo; riesgo principal: implementación incorrecta del aislamiento lógico puede resultar en cross-tenant data access, el fallo más grave en multi-tenancy y con las consecuencias legales más severas.
- **Modelo Bridge (tiered):** pool para tier Standard con límites de uso definidos, silo opcional para tier Enterprise con garantías adicionales — permite monetizar el nivel de aislamiento como feature diferencial del producto.
- **Noisy neighbor problem en pool:** mitigado con rate limiting por tenant en Redis (Token Bucket), priority queues por tier de servicio (Weighted Fair Queueing), y Resource

Quotas de Kubernetes por namespace de tenant.

- **Decisión de migración entre modelos:** comenzar con pool para los primeros 10-20 tenants y planificar la migración a bridge cuando algún tenant supere el 20% del consumo total de recursos del pool, o cuando el primer cliente Enterprise exija garantías contractuales de aislamiento.

Principio rector: El modelo de tenancy no es solo una decisión técnica — es un compromiso que impacta el modelo de precios del producto, los acuerdos de cumplimiento con los clientes, y la complejidad operacional del equipo de plataforma. Debe tomarse de manera explícita, con todos esos factores sobre la mesa.

Con el modelo de tenancy elegido, la sección siguiente profundiza en el control técnico más crítico de cualquier sistema multi-tenant de IA: el aislamiento de datos entre tenants, con especial atención a los índices vectoriales donde el riesgo de cross-tenant leakage es específico del dominio de IA.

Módulo 11 – Capítulo 04 – Sección 02

Aislamiento de datos entre tenants: cifrado por tenant y separación de índices vectoriales

El aislamiento de datos en sistemas multi-tenant de IA es más complejo que en aplicaciones CRUD convencionales porque los embeddings vectoriales son derivados de los datos originales, no los datos originales. Esta propiedad introduce un vector de riesgo que no existe en sistemas de bases de datos tradicionales: si un tenant embebe un documento confidencial y ese embedding termina en un índice compartido sin filtros de aislamiento correctamente implementados, una búsqueda semántica de un tenant diferente podría recuperar ese vector por similitud y el LLM podría sintetizar una respuesta que revela información del contenido original — aunque el segundo tenant nunca haya tenido acceso al documento fuente. El sistema estaría exfiltrando información mediante el espacio vectorial en lugar de mediante el acceso directo a los datos, un mecanismo de fuga que es difícil de detectar con las herramientas de auditoría convencionales.

La primera línea de defensa es el **cifrado por tenant con envelope encryption**. Cada tenant tiene una Customer Managed Key (CMK) gestionada en AWS KMS, Azure Key Vault, o HashiCorp Vault. Los datos del tenant en reposo — sus documentos, sus metadatos, sus configuraciones — están cifrados con un Data Encryption Key (DEK) que a su vez está cifrado con la CMK del tenant. Este esquema de doble cifrado tiene una propiedad operacional crítica: si el tenant decide revocar el acceso a la plataforma o si se detecta un incidente de seguridad, la revocación de la CMK del tenant hace inmediatamente inaccesibles todos sus datos almacenados en la plataforma, sin necesidad de borrar físicamente ningún dato — porque sin la clave, los datos son indistinguibles de ruido aleatorio.

La segunda línea de defensa, y la más específica del dominio de IA, es la **separación de índices vectoriales**. En Pinecone, esto se implementa mediante namespaces dedicados por tenant dentro del mismo índice, con la garantía de que una query vectorial que especifica un namespace nunca retorna resultados de otro namespace. En Weaviate, la multi-tenancy nativa (class per tenant) o las tenant-specific partitions proporcionan aislamiento a nivel de motor de búsqueda. En pgvector sobre PostgreSQL, el Row Level Security (RLS) garantiza que

cada query vectorial, por defecto, solo puede ver las filas donde el `tenant_id` coincide con el `tenant` del usuario autenticado — incluso si el desarrollador olvida incluir el filtro de `tenant_id` en la query de aplicación, el RLS actúa como red de seguridad automática.

La propagación del contexto del tenant — el `tenant_id` que identifica a qué tenant pertenece cada petición — debe ser un ciudadano de primera clase en el diseño del sistema multi-tenant. Se extrae del JWT del usuario autenticado en el API Gateway, se incluye en cada llamada entre microservicios mediante headers de contexto o como claim del JWT que viaja en cada petición, y se verifica en cada punto de acceso a datos sin excepciones. Un servicio que recibe un `tenant_id` de la petición HTTP pero no lo propaga a la llamada al servicio de vectorización produce un acceso a datos que no está aislado por tenant, independientemente de cuántas otras capas de aislamiento existan en el sistema.

Controles técnicos de aislamiento de datos

- **Envelope encryption por tenant:** cada tenant tiene un DEK cifrado con su CMK; la revocación de la CMK del tenant hace inaccesibles todos sus datos en la plataforma de manera inmediata, sin operaciones de borrado físico.
- **Namespaces en bases de datos vectoriales:** Pinecone namespaces por `tenant_id`, Weaviate multi-tenancy nativa con particiones por tenant, Qdrant collections separadas por tenant o payload filtering con `tenant_id` — cada opción con diferentes trade-offs de rendimiento y complejidad operacional.
- **Row Level Security en pgvector:** políticas RLS de PostgreSQL que filtran automáticamente las filas por `tenant_id` extraído del JWT del usuario autenticado, actuando como red de seguridad adicional ante errores de aplicación.
- **Tenant context propagation:** el `tenant_id` se extrae del JWT en el API Gateway y se propaga mediante headers de contexto en cada llamada entre microservicios, verificado en cada punto de acceso a datos con middleware de validación.
- **Auditoría de acceso a datos:** logging de cada query vectorial con el `tenant_id` del solicitante, el namespace o colección consultada, y los document IDs retornados, retenido mínimo 12 meses para auditoría forense en caso de incidente.

Para recordar: El aislamiento de datos en multi-tenancy de IA debe implementarse como defensa en profundidad: cifrado a nivel de almacenamiento + separación de índices a nivel de base de datos vectorial + `tenant_id` verificado en cada query a nivel de aplicación. Ninguna

capa individual es suficiente. La intersección de las tres capas es lo que produce un aislamiento robusto.

Este control técnico de aislamiento de datos se complementa con el control operacional de la sección siguiente: el rate limiting por tenant que evita que el comportamiento de un tenant afecte la experiencia de los demás, independientemente del nivel de aislamiento de datos.

Módulo 11 – Capítulo 04 – Sección 03

Rate limiting y fairness por tenant: evitar que un tenant afecte a otros

El problema del "noisy neighbor" en plataformas multi-tenant de IA tiene una dimensión específica que no existe en las plataformas web convencionales: la varianza en el consumo de recursos por petición es extremadamente alta. En una plataforma web tradicional, cada petición HTTP consume aproximadamente el mismo tiempo de cómputo. En una plataforma de IA, una petición que clasifica un texto corto puede consumir 200 tokens en 200ms, mientras una petición que analiza un contrato de 50 páginas consume 50.000 tokens en 45 segundos. Esta varianza hace que el impacto de un tenant que ejecuta múltiples peticiones de alta complejidad simultáneamente sea cualitativamente diferente al de uno que hace peticiones de baja complejidad a la misma tasa.

El rate limiting por tenant en sistemas de IA debe operar en múltiples dimensiones simultáneamente para capturar adecuadamente esta varianza. **Tokens por minuto (TPM)** controla el consumo de contexto del LLM y traduce directamente a costo: un tenant que consume 1 millón de tokens en un minuto no debería poder hacerlo si su plan solo permite 100.000 TPM. **Requests por segundo (RPS)** controla la frecuencia de consultas a la API, independientemente del tamaño de cada petición: previene que un tenant abra 500 conexiones simultáneas aunque cada una sea pequeña. **Unidades de vectorización por hora** controla la carga de indexación en la base de datos vectorial: un job de indexación masiva de documentos puede saturar el servicio de embeddings para todos los tenants si no está limitado.

La implementación técnica del rate limiting distribuido — necesaria porque la plataforma opera con múltiples réplicas del servicio de orquestación — requiere un almacén de estado compartido. Redis es el estándar en este rol, y el algoritmo Token Bucket implementado con scripts Lua atómicos (para garantizar atomicidad en el check-and-decrement del contador) es el mecanismo más usado. El Token Bucket permite burst de corta duración — un tenant puede exceder su límite momentáneamente si acumuló tokens en períodos de baja actividad — mientras aplica el límite de largo plazo. Esta flexibilidad es importante para plataformas

enterprise donde los usuarios tienen picos de uso predecibles (inicio de jornada laboral, fin de mes para procesos financieros) que justifican tolerar bursts sin penalizar al tenant.

La **fairness entre tenants** va más allá del rate limiting individual: requiere que el scheduler de la plataforma garantice que un tenant que consume consistentemente poco no resulte penalizado cuando el sistema está bajo carga por culpa de otros tenants. El Weighted Fair Queueing (WFQ) implementado a nivel de las colas de procesamiento garantiza que cada tenant recibe su cuota proporcional de capacidad de procesamiento incluso cuando el sistema está saturado, asignando más capacidad a los tenants de alto tier (Enterprise, Pro) sin dejar de servir a los de bajo tier (Standard, Free).

Nota del Arquitecto: Publicar los límites de rate limiting en el portal de desarrolladores desde el primer día, junto con las métricas de consumo actual del tenant, evita el escenario más frustrante del multi-tenancy: que un tenant descubra sus límites en producción durante un pico crítico de negocio, a las 11pm del último día del trimestre, cuando no hay nadie disponible para aumentar el límite. La transparencia sobre los límites crea confianza; descubrir los límites en producción la destruye.

Aspectos técnicos del rate limiting en IA multi-tenant

- **Token bucket distribuido en Redis:** implementado con RedisLimiter o redis-cell (módulo de Redis para rate limiting), con scripts Lua atómicos que garantizan que el check-and-decrement es una operación atómica incluso con múltiples réplicas del servicio consultando el mismo límite.
- **Límites en múltiples dimensiones:** TPM (tokens per minute) para costo de LLM, RPM (requests per minute) para frecuencia de API, y concurrent_requests para saturación de conexiones simultáneas, con configuración independiente por tenant y por tipo de operación.
- **Priority queues por tier de servicio:** colas de procesamiento con prioridad alta (tenants Enterprise), media (tenants Pro), y baja (tenants Standard), procesadas con Weighted Fair Queueing para garantizar que los tenants de menor tier reciben respuesta incluso durante picos de alta demanda.
- **Adaptive rate limiting:** ajuste dinámico de los límites por tenant basado en la capacidad disponible del sistema — en períodos de baja carga, los límites pueden relajarse temporalmente; en períodos de alta carga, se aplican más estrictamente.

- **Respuestas de rate limit apropiadas:** HTTP 429 con headers Retry-After y X-RateLimit-Reset que proporcionan la información necesaria para que los clientes implementen backoff exponencial sin polling agresivo que agrave la saturación.
-

Buena práctica: Los límites de rate limiting deben ser visibles para los tenants desde el primer día — publicarlos en el portal de desarrolladores con las métricas de consumo actual evita que los tenants descubran los límites en producción durante picos críticos de negocio. La transparencia sobre los límites es una función del producto, no solo de la documentación.

Con el aislamiento de datos y el rate limiting establecidos, la sección siguiente aborda la dimensión de personalización: cómo cada tenant puede configurar el comportamiento del sistema de IA para su caso de uso específico sin requerir cambios en el código de la plataforma.

Módulo 11 – Capítulo 04 – Sección 04

Personalización por tenant: modelos, prompts y comportamientos específicos por cliente

La personalización en plataformas de IA multi-tenant va sustancialmente más allá de las personalizaciones superficiales que los SaaS convencionales ofrecen — el logo, los colores, el nombre de la instancia. En una plataforma de IA enterprise, la personalización por tenant determina qué modelo de LLM se usa (con implicaciones de costo, calidad, y cumplimiento), cómo se comporta el asistente (su persona, sus instrucciones base, sus restricciones temáticas), qué documentos alimentan su base de conocimiento, y qué herramientas y capacidades tiene disponibles. Dos tenants de la misma plataforma pueden experimentar sistemas de IA cualitativamente distintos no porque la plataforma subyacente sea diferente, sino porque la configuración por tenant crea sistemas con comportamientos diferenciados sobre la misma infraestructura.

El componente central que hace esto posible es el **tenant configuration service**: un servicio que almacena en base de datos (PostgreSQL con JSONB o DynamoDB) la configuración específica de cada tenant y la expone mediante una API que el servicio de orquestación consulta en cada petición. La consulta directa en cada petición introduciría latencia inaceptable, por lo que la configuración del tenant se cachea en Redis con un TTL de 30-60 segundos. Este TTL es la latencia máxima con la que un cambio de configuración del tenant — por ejemplo, actualizar el system prompt — se propaga al sistema en producción sin requerir un despliegue.

El **system prompt por tenant** es el mecanismo de personalización de mayor impacto: define la persona del asistente (nombre, tono, nivel de formalidad), sus instrucciones de comportamiento específicas del negocio del tenant (responder solo sobre productos propios del cliente, siempre referir a un humano cuando no se tiene confianza en la respuesta, usar la terminología interna del sector), y las restricciones temáticas (no discutir competidores, no dar consejos legales o médicos, restringir las respuestas al dominio del negocio). Almacenado en el prompt registry versionado — el mismo que se describe en el Capítulo 05 sobre LLMOps

— con la capacidad de hacer rollback inmediato a la versión anterior y de ejecutar A/B tests entre versiones del system prompt para un tenant sin afectar a otros.

La **selección de modelo por tenant** abre la dimensión más compleja de la personalización: algunos tenants enterprise tienen contratos propios con proveedores de LLM (una empresa farmacéutica puede tener un acuerdo de BAA con Microsoft Azure OpenAI para datos HIPAA), otros requieren modelos self-hosted por restricciones de residencia de datos (datos que no pueden salir de la UE requieren modelos desplegados en infraestructura europea propia), y otros están dispuestos a pagar más por el modelo premium para sus casos de uso más críticos. El sistema de routing de modelos debe soportar esta configuración por tenant de manera dinámica y con fallback configurable.

El **fine-tuning por tenant con LoRA adapters** es la forma más avanzada de personalización: un adaptador LoRA (Low-Rank Adaptation) entrenado sobre los documentos y el vocabulario específico del tenant se carga dinámicamente sobre el modelo base cuando llega una petición de ese tenant, adaptando el comportamiento del modelo a su dominio específico sin requerir un modelo completamente separado. vLLM soporta el cargado dinámico de adaptadores LoRA con latencia de carga de milisegundos, haciendo esta capacidad prácticamente transparente para el usuario final.

Mecanismos de personalización por tenant

- **System prompt por tenant:** almacenado en el prompt registry versionado con capacidad de A/B testing entre versiones para un mismo tenant y rollback inmediato a la versión anterior sin despliegue de código.
- **Selección de modelo por tenant:** routing configurable que permite elegir entre GPT-4o, Claude Sonnet, Gemini Pro, o modelos open-source self-hosted, con fallback configurable si el modelo primario no está disponible o supera el timeout.
- **Índices RAG por tenant:** cada tenant tiene su colección en la base de datos vectorial con sus propios documentos, chunking strategy configurada, y reranker específico del dominio — completamente aislada de otros tenants (ver Sección 02 de este capítulo para el mecanismo de aislamiento).
- **LoRA adapters por tenant:** fine-tuning ligero entrenado con los datos del tenant que personaliza el vocabulario y los patrones de respuesta, cargado dinámicamente con vLLM's LoRA adapter loading sin tiempo de carga perceptible para el usuario.
- **Feature flags por tenant:** capacidades habilitadas o deshabilitadas por configuración (búsqueda web, ejecución de código, acceso a herramientas externas) según el plan del

tenant o sus requisitos de cumplimiento, sin modificar el código de la plataforma.

Idea central: La personalización por tenant debe ser un sistema de configuración dinámico y auditable — no hardcodeado por tenant en el código fuente — para que los cambios de configuración sean desplegados sin modificar el código, reversibles sin rollback de infraestructura, y auditables para cumplimiento. El código debe ser el mismo para todos los tenants; la configuración es lo que los diferencia.

La sección siguiente aborda la dimensión económica de la escalabilidad multi-tenant: cómo el sistema crece para acomodar nuevos tenants sin degradar la experiencia de los existentes, y cómo se atribuyen los costos a cada tenant para habilitar el modelo de negocio de la plataforma.

Módulo 11 – Capítulo 04 – Sección 05

Escalabilidad y cost allocation: crecer con nuevos tenants sin degradación de servicio

Una plataforma multi-tenant que funciona bien con 5 tenants y degrada con 50 no es una plataforma de producción: es un piloto exitoso con una trampa de escala. La diferencia entre las dos situaciones es que las decisiones de arquitectura que permiten escalar sin degradación deben tomarse antes de que la degradación aparezca, no en respuesta a ella. Diseñar retroactivamente un sistema de auto-scaling y cost allocation sobre una plataforma que ya tiene 50 tenants activos en producción es significativamente más costoso y arriesgado que haberlo diseñado desde el inicio.

La **escalabilidad horizontal** en plataformas multi-tenant de IA opera con señales más complejas que el CPU utilization típico que dispara el Horizontal Pod Autoscaler convencional. El KEDA (Kubernetes Event-Driven Autoscaling) es el componente que permite escalar basándose en métricas de negocio y de carga de IA: el lag del consumer group de Kafka por topic, el número de requests pendientes en la cola de Redis, o métricas personalizadas como el número de usuarios activos por tenant en los últimos 5 minutos. La ventaja de estas métricas es que son predictivas de la demanda futura, no reactivas a la saturación actual: cuando el consumer lag de Kafka empieza a crecer, el auto-scaling puede añadir réplicas antes de que la latencia de procesamiento se degrade perceptiblemente.

El **cost allocation por tenant** es el componente que transforma la plataforma de IA en un negocio. Sin cost allocation, la plataforma opera como un centro de costo opaco donde solo se conoce el costo total pero no qué tenant lo genera ni qué operaciones son las más costosas. Con cost allocation, cada petición de inferencia registra el tenant_id, el modelo utilizado, el número de tokens de entrada y salida (que determina el costo de la llamada al LLM externo), los recursos de cómputo consumidos en la inferencia (GPU-seconds para modelos self-hosted), y el número de operaciones vectoriales ejecutadas para el RAG. Con estos datos consolidados en un sistema de métricas analíticas (ClickHouse, BigQuery, o Prometheus con Thanos para retención larga), el equipo de plataforma puede generar informes de costo por

tenant con granularidad diaria, identificar los tenants más costosos para la plataforma, y calcular el margen de cada tier de servicio.

El cost allocation tiene además una función interna crítica en plataformas enterprise que sirven a múltiples unidades de negocio: el **showback y chargeback**. El showback muestra a cada unidad de negocio cuánto consume de la plataforma de IA sin cobrarles directamente; el chargeback les cobra internamente esa cantidad cargándola a su presupuesto departamental. Ambos mecanismos generan incentivos para el uso eficiente de la plataforma: cuando el equipo de marketing sabe que sus 500 consultas diarias al asistente de IA le cuestan 150 USD/mes al presupuesto del departamento, tiene incentivos para entender si cada una de esas consultas está generando valor de negocio proporcional.

Componentes de escalabilidad y cost allocation

- **KEDA para auto-scaling reactivo:** escalado horizontal de pods de inferencia y orquestación basado en métricas de Kafka consumer lag, Redis queue length, o métricas personalizadas de Prometheus, con tiempo de escaldo de 30-60 segundos desde que se detecta el incremento de carga.
- **Chargeback por token consumido:** registro en ClickHouse de cada inferencia con tenant_id, model_id, input_tokens, output_tokens, latency_ms, y costo calculado, con retención de 90 días para análisis y 12 meses para facturación.
- **Resource Quotas por tenant en Kubernetes:** LimitRanges y ResourceQuotas por namespace de tenant que garantizan que un tenant no puede consumir más de su cuota asignada de CPU y memoria del cluster compartido.
- **Dashboard de costos por tenant:** Grafana conectado a ClickHouse mostrando costo diario, tendencia mensual, desglose por modelo y operación, y alerta cuando el gasto proyectado supera el presupuesto asignado al tenant o a la unidad de negocio.
- **Optimización automática del pool:** identificación de los tenants con menor actividad en las últimas 24 horas para desprovisionamiento temporal de recursos dedicados y liberación de capacidad del pool para tenants activos, con re-aprovisionamiento automático cuando el tenant vuelve a ser activo.

Para recordar: El cost allocation por tenant no es una feature a implementar cuando haya tiempo libre: sin él, es imposible tomar decisiones informadas de precio del producto,

identificar tenants no rentables que están subsidiados por la plataforma, o justificar la inversión en infraestructura ante el liderazgo con datos de retorno real.

La sección de cierre integra los cinco temas del capítulo — modelos de tenancy, aislamiento de datos, rate limiting, personalización, y cost allocation — en el argumento central sobre por qué el multi-tenancy en IA requiere rigor técnico en cada capa del stack.

Módulo 11 – Capítulo 04 – Sección 06

Cierre: el multi-tenancy en IA exige aislamiento técnico riguroso en cada capa del stack

Una plataforma multi-tenant de IA que no puede garantizar el aislamiento riguroso de datos entre tenants no puede operar en sectores regulados, no puede firmar los DPAs que los clientes enterprise exigen, y expone a la organización a responsabilidades legales en caso de un incidente de cross-tenant data access. Esta restricción no es negociable: a diferencia de otros aspectos del diseño donde los trade-offs son continuos y reversibles, el aislamiento de datos en multi-tenancy es una propiedad que el sistema tiene o no tiene — y cuando no la tiene, el descubrimiento del fallo ocurre típicamente en el peor momento posible.

El conjunto de controles técnicos cubiertos en este capítulo — los tres modelos de tenancy con sus trade-offs explícitos, el cifrado por tenant con CMKs revocables, la separación de índices vectoriales mediante namespaces y Row Level Security, el rate limiting distribuido con Redis y Token Bucket, la personalización dinámica con el tenant configuration service, y el cost allocation por inferencia — constituyen el mínimo técnico para operar una plataforma multi-tenant de IA con datos sensibles en un contexto enterprise. La conexión con el Capítulo 06 de este módulo es directa: cuando se implemente el RAG empresarial multi-tenant, el aislamiento de índices vectoriales descrito aquí es exactamente el mecanismo que garantiza que el permission-aware retrieval del RAG funciona correctamente — los documentos de un tenant solo pueden ser recuperados en el contexto de ese tenant.

El aislamiento no es un estado binario sino un espectro, y cada capa adicional de aislamiento reduce el riesgo pero aumenta el costo operacional y la complejidad del sistema. El modelo Bridge — pool para tenants estándar, silo para tenants Enterprise — es la solución que equilibra el espectro de manera pragmática para la mayoría de los casos enterprise: permite operar eficientemente el 80% del catálogo de tenants mientras ofrece las garantías de aislamiento más fuertes para el 20% que las requiere contractualmente. La decisión sobre el nivel de aislamiento apropiado debe tomarse explícitamente, documentarse en la arquitectura de referencia del sistema, y revisarse periódicamente a medida que crece el número de tenants y cambia su perfil de cumplimiento.

El siguiente capítulo lleva el foco al ciclo de vida operacional de los LLMs: cómo se evalúan continuamente, cómo se versionan y despliegan los prompts como artefactos de ingeniería, y cómo se planifican y ejecutan los rollbacks cuando una actualización del modelo degrada la calidad del sistema.

"La seguridad no es un producto sino un proceso: el aislamiento multi-tenant debe diseñarse, implementarse, monitorearse y auditarse continuamente, porque los atacantes no descansan." — Bruce Schneier, criptógrafo y experto en seguridad informática